
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 8 — Reinforcement Learning

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	4
1 Thuật toán học tăng cường (Reinforcement Learning Algorithms)	5
1.1 Học tăng cường là gì?	5
1.2 Hai loại thuật toán RL	5
1.3 Thuật toán model-based	5
1.3.1 Lập trình động (Dynamic Programming)	6
1.3.2 Monte Carlo Tree Search (MCTS)	6
1.4 Thuật toán model-free	6
1.5 So sánh model-based và model-free	7
2 Khai thác và khám phá (Exploitation and Exploration)	7
2.1 Khai thác (Exploitation)	8
2.2 Khám phá (Exploration)	8
2.3 Lựa chọn hành động (Action Selection)	8
2.4 Đánh đổi khai thác và khám phá	9
2.5 Cân bằng khai thác và khám phá	9
2.5.1 Epsilon-Greedy	9
2.5.2 Multi-Armed Bandit	10
2.5.3 Upper Confidence Bound (UCB)	10
3 Q-Learning	11
3.1 Thành phần chính	11
3.2 Cách Q-Learning hoạt động	11
3.2.1 Temporal Difference (TD)	12
3.2.2 Phương trình Bellman	12
3.3 Thuật toán Q-Learning	12
3.4 Ưu, nhược điểm và ứng dụng	13
4 Thuật toán REINFORCE	14
4.1 Khái niệm chính	14
4.2 Cách REINFORCE hoạt động	14
4.2.1 Lấy mẫu episode (Episode Sampling)	15
4.2.2 Quỹ đạo (Trajectory)	15
4.2.3 Return G_t	15
4.2.4 Policy gradient	15
4.2.5 Cập nhật policy	15
4.3 Ưu và nhược điểm	16
5 SARSA (State-Action-Reward-State-Action)	16
5.1 Ví dụ robot trong mê cung	17
5.2 Phương trình cập nhật	17
5.3 Các thành phần cốt lõi	17
5.4 Cách SARSA hoạt động	18
5.5 SARSA so với Q-learning	18

6	Phương pháp Actor–Critic	19
6.1	Hai thành phần chính	19
6.2	Cách hoạt động từng bước	20
6.3	Ưu điểm	21
6.4	Thách thức	21
6.5	Các biến thể tiêu biểu	21
6.6	Thuật toán A2C	22
6.7	Thuật toán A3C	22
6.8	A2C so với A3C	23
7	Phương pháp Monte Carlo trong RL	23
7.1	Khái niệm then chốt	23
7.2	Đánh giá chính sách Monte Carlo	24
7.3	On-policy và off-policy	24
7.4	Điều khiển Monte Carlo (MC control)	25
7.5	Ứng dụng	25
7.6	Hạn chế	26
8	Học chênh lệch thời gian (Temporal Difference)	26
8.1	Tham số $\alpha, \gamma, \varepsilon$	26
8.2	TD trong AI và học máy	27
8.3	Thuật toán TD(λ)	27
8.4	TD(0)	27
8.5	TD(1)	28
8.6	TD so với Q-learning	28
8.7	Sai số TD (TD error)	28
8.8	Lợi ích	29
8.9	Thách thức	29

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Reinforcement Learning: https://www.tutorialspoint.com/machine_learning/machine_learning_reinforcement_learning_algorithms.htm.

Cách dùng

- Đọc sau phần Dimensionality Reduction.
- Chạy code từ thư mục 8.ReinforcementLearning/.

Đặng Tấn Quý

1 Thuật toán học tăng cường (Reinforcement Learning Algorithms)

Mục tiêu học tăng cường

Thuật toán học tăng cường huấn luyện **agent** ra quyết định tối ưu trong môi trường. Mục tiêu là **tối đa hóa phần thưởng tích lũy** (cumulative reward) theo thời gian. Các thuật toán phổ biến gồm Q-learning, policy gradient, Monte Carlo, ...

1.1 Học tăng cường là gì?

Reinforcement Learning (RL)

Học tăng cường là hướng học máy trong đó **agent** (thực thể phần mềm) được huấn luyện để hiểu môi trường bằng cách **thực hiện hành động** và theo dõi kết quả. Hành động tốt $\Rightarrow$ phản hồi tích cực; hành động xấu $\Rightarrow$ phản hồi tiêu cực. Cách tiếp cận lấy cảm hứng từ việc động vật học từ kinh nghiệm: quyết định dựa trên **hậu quả** hành động.

1.2 Hai loại thuật toán RL

Model-based và model-free

Thuật toán RL chia thành hai nhóm chính: **model-based** và **model-free**. Điểm khác biệt nằm ở cách xác định **chính sách tối ưu** π .

Model-Based Reinforcement Learning

Agent **xây mô hình** môi trường và **dự đoán** kết quả của hành động ở các trạng thái khác nhau. Sau khi có mô hình, agent dùng nó để lập chiến lược và dự báo tương lai mà **không cần** tương tác trực tiếp liên tục với môi trường. Cách này có thể **tăng hiệu quả** ra quyết định vì không hoàn toàn phụ thuộc thử-sai.

Model-Free Reinforcement Learning

Agent **không** duy trì mô hình môi trường. Thay vào đó, học **chính sách** hoặc **hàm giá trị** qua tương tác trực tiếp với môi trường.

1.3 Thuật toán model-based

Các thuật toán tối ưu và điều khiển model-based

1. Dynamic Programming (Lập trình động)
2. Monte Carlo Tree Search (MCTS)

1.3.1 Lập trình động (Dynamic Programming)

Dynamic Programming

Lập trình động là khung toán học giải bài toán phức tạp, đặc biệt trong **ra quyết định** và **điều khiển**. Khi agent **biết đầy đủ** môi trường (mô hình hoàn hảo), có thể dùng các thuật toán DP trong RL để tìm chính sách tối ưu.

Value Iteration — Lập giá trị

Value Iteration là thuật toán DP tính **chính sách tối ưu** bằng cách ước lượng giá trị mỗi trạng thái với giả định agent theo policy tối ưu. Cập nhật dựa trên phương trình Bellman:

$$V(s) = \max_a \sum_{s',r} P(s',r | s,a) (R(s,a,s') + \gamma V(s'))$$

trong đó $P(s',r | s,a)$ là xác suất chuyển sang s' và nhận r ; γ là hệ số chiết khấu.

Policy Iteration — Lập chính sách

Policy Iteration là quy trình hai bước tìm đồng thời **hàm giá trị tối ưu** V^π và **chính sách tối ưu** π :

1. **Policy Evaluation (Đánh giá chính sách)**: với policy cho trước, tính hàm giá trị mọi trạng thái bằng phương trình Bellman.
2. **Policy Improvement (Cải thiện chính sách)**: dùng hàm giá trị hiện tại, cải thiện policy bằng cách chọn hành động **tối đa hóa** phần thưởng kỳ vọng.

Hai bước **luân phiên** cho đến khi policy hội tụ về policy tối ưu.

1.3.2 Monte Carlo Tree Search (MCTS)

MCTS

Monte Carlo Tree Search là thuật toán **tìm kiếm heuristic** dùng **cây** để khám phá các hành động và trạng thái có thể. MCTS đặc biệt hữu ích cho ra quyết định trong môi trường **phức tạp**.

1.4 Thuật toán model-free

Danh sách thuật toán model-free chính

1. **Monte Carlo Learning** — ước lượng hàm giá trị và policy từ **kinh nghiệm thực**, lấy trung bình qua nhiều episode thay vì dùng mô hình động học môi trường.
2. **Temporal Difference (TD) Learning** — cập nhật giá trị **tăng dần** sau mỗi hành động và mỗi reward (khác Monte Carlo chỉ cập nhật sau khi episode kết thúc); phù hợp ra quyết định online.
3. **SARSA** — thuật toán **on-policy**, model-free, học hàm giá trị hành động $Q(s,a)$;

tên viết tắt State–Action–Reward–State–Action; cập nhật theo hành động agent **thực sự** thực hiện.

4. **Q-Learning** — thuật toán **off-policy**, model-free, học $Q^*(s, a)$: phần thưởng kỳ vọng tối đa cho cặp trạng thái–hành động; mục tiêu tìm policy tối ưu qua hàm Q tối ưu.
5. **Policy Gradient Optimization** — tối ưu **trực tiếp** tham số policy thay vì chỉ học hàm giá trị; REINFORCE là một dạng policy gradient dựa trên Monte Carlo.

1.5 So sánh model-based và model-free

Bảng so sánh Model-Based RL và Model-Free RL

Tiêu chí	Model-Based RL	Model-Free RL
Quá trình học	Ban đầu học mô hình động học môi trường, dùng mô hình dự đoán hành động tương lai.	Dựa hoàn toàn vào thử-sai; học policy hoặc hàm giá trị trực tiếp từ chuyển trạng thái và reward quan sát được.
Hiệu quả mẫu	Có thể đạt sample efficiency cao hơn vì mô phỏng nhiều tương tác bằng mô hình đã học.	Cần nhiều tương tác thực hơn với môi trường để tìm policy tối ưu.
Độ phức tạp	Phức tạp hơn: phải học và duy trì mô hình môi trường chính xác.	Tương đối đơn giản hơn vì không cần huấn luyện mô hình riêng.
Sử dụng môi trường	Chủ động xây mô hình để dự đoán kết quả và hành động tiếp theo.	Không xây mô hình; phụ thuộc trực tiếp vào kinh nghiệm trước đó.
Khả năng thích nghi	Có thể thích nghi khi trạng thái môi trường thay đổi (nhờ mô hình cập nhật).	Có thể chậm thích nghi vì dựa nhiều vào kinh nghiệm cũ.
Yêu cầu tính toán	Thường cần tài nguyên tính toán lớn hơn (học và duy trì mô hình).	Thường ít tốn tính toán hơn, tập trung học trực tiếp từ trải nghiệm.

2 Khai thác và khám phá (Exploitation and Exploration)

Hai chiến lược trong RL

Trong học máy, **khám phá (exploration)** là cho phép agent tìm hiểu **đặc điểm mới** của môi trường; **khai thác (exploitation)** là bắt agent **bám** kiến thức đã có. Nếu agent chỉ khai thác, dễ **kẹt** ở giải pháp cục bộ. Nếu chỉ khám phá, có thể **không bao giờ** tìm được policy tốt — đây là **dilemma khai thác–khám phá**.

2.1 Khai thác (Exploitation)

Exploitation

Khai thác là chiến lược RL trong đó agent dùng **kiến thức hiện có** để ra quyết định ở một trạng thái nhằm **tối đa hóa phần thưởng kỳ vọng**. Mục tiêu: tận dụng những gì đã biết về môi trường để đạt **kết quả tốt nhất**.

Các khía cạnh chính của khai thác

- **Tối đa hóa reward**: chọn hành động theo giá trị và reward đã học để đạt kết quả cao nhất.
- **Hiệu quả ra quyết định**: tập trung hành động reward cao, giảm chi phí tính toán cho khám phá.
- **Quản lý rủi ro**: rủi ro thấp vì ưu tiên hành động đã thử, giảm bất định so với lựa chọn ít quen thuộc.

2.2 Khám phá (Exploration)

Exploration

Khám phá là hành động giúp agent **thu thập kiến thức** về môi trường hoặc mô hình. Quá trình khám phá chọn hành động có **kết quả không chắc chắn** để thu thập thông tin về trạng thái và reward mà hành động sẽ mang lại.

Các khía cạnh chính của khám phá

- **Thu thập thông tin**: thử hành động mới ở trạng thái để cải thiện hiểu biết về môi trường hoặc mô hình.
- **Giảm bất định**: giảm độ không chắc chắn về reward và chuyển trạng thái của các hành động chưa thử nhiều.
- **Phủ không gian trạng thái**: với không gian trạng thái lớn hoặc liên tục, khám phá đảm bảo thăm đủ vùng, tránh học **lệch** vì chỉ dựa trên ít trải nghiệm.

2.3 Lựa chọn hành động (Action Selection)

Mục tiêu huấn luyện agent

Mục tiêu RL là dạy agent **cách hành xử** dưới nhiều trạng thái. Trong huấn luyện, agent học hành động qua các cách chọn hành động như: **greedy**, **epsilon-greedy**, **Upper Confidence Bound (UCB)**, ...

2.4 Đánh đổi khai thác và khám phá

Exploration vs. Exploitation trade-off

Ý tưởng dùng kiến thức hiện có so với thử hành động ngẫu nhiên gọi là đánh đổi khai thác–khám phá. Khi khám phá, agent có thể **cải thiện** kiến thức theo thời gian. Khi khai thác, agent nhận **reward cao hơn ngay lập tức**. Agent không thể làm cả hai tối đa cùng lúc $\Rightarrow$ cần **cân bằng**.

Phân bổ tài nguyên (thời gian/thử nghiệm) nên phụ thuộc nhu cầu của cả hai hướng, **luân phiên** theo trạng thái hiện tại và độ phức tạp bài học.

2.5 Cân bằng khai thác và khám phá

Các kỹ thuật cân bằng

- Epsilon-Greedy Action Selection
- Multi-Armed Bandit Frameworks
- Upper Confidence Bound (UCB)

2.5.1 Epsilon-Greedy

Epsilon-Greedy Action Selection

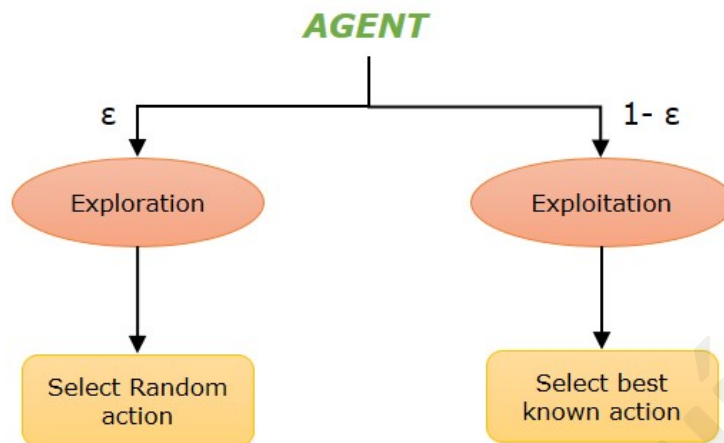
Trong RL, agent thường chọn hành động theo reward. Cách tham lam (greedy): luôn chọn hành động **tối ưu** để tối đa reward ở trạng thái hiện tại. **Epsilon-greedy** kết hợp **khai thác** (dùng kiến thức cũ) và **khám phá** (tìm lựa chọn mới).

Cơ chế epsilon-greedy

Phương pháp thường chọn hành động có **reward kỳ vọng cao nhất**. Mục tiêu: cân bằng khám phá và khai thác. Với xác suất nhỏ ε , agent **khám phá** (chọn ngẫu nhiên hoặc khác greedy) thay vì chỉ khai thác những gì đã học.

Minh họa epsilon-greedy

Epsilon Greedy Action Selection



2.5.2 Multi-Armed Bandit

Multi-Armed Bandit

Khung **multi-armed bandit** cung cấp nền tảng hình thức quản lý cân bằng khai thác–khám phá trong bài toán ra quyết định **tuần tự**. Các thuật toán trong khung này phân tích đánh đổi dựa trên **cơ chế reward** và hoàn cảnh cụ thể (ví dụ mỗi “cánh tay” máy đánh bạc cho reward khác nhau).

2.5.3 Upper Confidence Bound (UCB)

Upper Confidence Bound

UCB là thuật toán phổ biến cân bằng khai thác–khám phá, dựa trên nguyên tắc **lạc quan trước bất định** (optimism in the face of uncertainty). UCB chọn hành động tối ưu hóa **cận trên tin cậy** (upper confidence bound) của reward kỳ vọng: xét cả **reward trung bình** và **độ không chắc chắn/biến thiên** của hành động đó.

Gợi ý thực hành

ε lớn $\Rightarrow$ khám phá nhiều, học chậm nhưng tránh kẹt cục bộ. ε nhỏ $\Rightarrow$ khai thác mạnh, hội tụ nhanh nhưng dễ bỏ sót hành động tốt hơn. UCB và bandit phù hợp khi cần cân bằng có **cơ sở lý thuyết** hơn so với random ε đơn thuần.

3 Q-Learning

Q-Learning

Q-learning là thuật toán RL **dựa trên giá trị (value-based)**: mô hình học và cải thiện dần qua thời gian bằng cách chọn **hành động đúng**. Hành động đúng được coi là **phần thưởng**; hành động sai là **hình phạt**.

Q-Learning trong bối cảnh RL

Học tăng cường: agent học theo thời gian để ra quyết định đúng trong môi trường bằng **tương tác liên tục**. Các tình huống agent gặp gọi là **trạng thái (states)**. Ở mỗi trạng thái, agent thực hiện hành động từ tập hành động khả dụng và nhận reward hoặc penalty. Theo thời gian, agent học **tối đa hóa** các reward này. Q-learning dùng **giá trị Q** (action values) để cải thiện hành vi lặp lại.

3.1 Thành phần chính

Các thành phần Q-Learning

Q-learning hoạt động qua quy trình lặp: agent khám phá môi trường và cập nhật mô hình liên tục.

- **Agent**: thực thể thực hiện tác vụ trong môi trường.
- **States**: biến mô tả **tình huống hiện tại** của agent trong môi trường.
- **Actions**: hành vi của agent tại một trạng thái cụ thể.
- **Rewards**: phản hồi tích cực hoặc tiêu cực cho hành động của agent.
- **Episodes**: một episode kết thúc khi agent đến điểm **không thể** thực hiện thêm hành động (trạng thái kết thúc).
- **Q-values**: đại lượng đánh giá **chất lượng** một hành động tại một trạng thái.

3.2 Cách Q-Learning hoạt động

Học qua thử-sai

Q-Learning học kết quả của hành động agent thực hiện trong môi trường qua trải nghiệm thử-sai. Quá trình gồm học hành vi tối ưu bằng hàm giá trị hành động tối ưu — **hàm Q**. Có hai cách (công thức) xác định giá trị Q phổ biến: **Temporal Difference** và **phương trình Bellman**.

3.2.1 Temporal Difference (TD)

Cập nhật Temporal Difference

Phương trình TD xác định Q-value bằng cách so sánh trạng thái–hành động **hiện tại** và **trước đó**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

- s : trạng thái hiện tại của agent.
- a : hành động hiện tại (từ bảng Q).
- s' : trạng thái kế tiếp (có thể là trạng thái kết thúc).
- a' : hành động tốt nhất kế tiếp theo ước lượng Q hiện tại.
- r : reward quan sát từ môi trường sau hành động hiện tại.
- $\gamma \in (0, 1]$: hệ số chiết khấu phần thưởng tương lai.
- α : bước học (learning rate) cập nhật ước lượng $Q(s, a)$.

3.2.2 Phương trình Bellman

Phương trình Bellman trong Q-Learning

Richard Bellman đề xuất (1957) cách ra quyết định tối ưu bằng đệ quy. Trong Q-learning, phương trình Bellman xác định giá trị trạng thái và so sánh vị trí tương đối; trạng thái có giá trị cao nhất được coi là tối ưu. Dạng Bellman cho Q-value:

$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$$

- $Q(s, a)$: reward kỳ vọng khi thực hiện a ở s .
- $r(s, a)$: reward thu được khi thực hiện a ở s .
- γ : hệ số chiết khấu, đánh giá tầm quan trọng reward tương lai.
- $\max_{a'} Q(s', a')$: giá trị Q lớn nhất ở trạng thái kế tiếp s' trên mọi hành động khả dụng.

3.3 Thuật toán Q-Learning

Bảng Q (Q-table)

Bảng Q lưu reward gắn với **hành động tối ưu** cho từng trạng thái trong môi trường. Agent khám phá môi trường và cập nhật bảng Q theo reward nhận được.

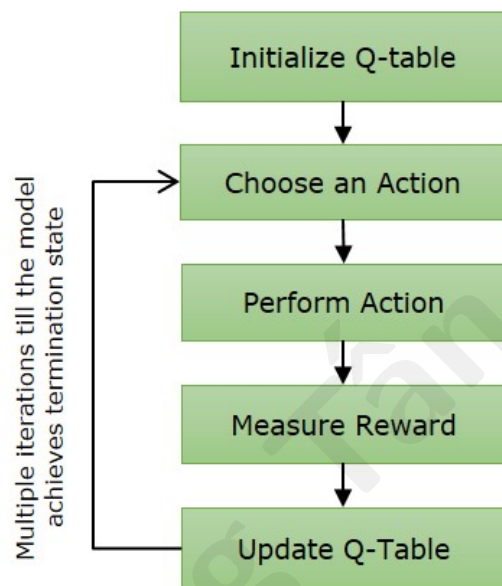
Các bước thuật toán Q-Learning

1. **Khởi tạo bảng Q**: khởi tạo bảng Q để theo dõi tiến độ hành động ở các trạng thái.

2. **Quan sát:** agent quan sát trạng thái hiện tại của môi trường.
3. **Hành động:** agent chọn và thực hiện hành động; sau đó kiểm tra hành động có hữu ích trong môi trường hay không.
4. **Cập nhật:** cập nhật bảng Q bằng kết quả (công thức TD hoặc Bellman).
5. **Lặp:** lặp bước 2–4 đến khi đạt **trạng thái kết thúc**.

Sơ đồ thuật toán Q-Learning

Q-Learning Algorithm



3.4 Ưu, nhược điểm và ứng dụng

Ưu điểm Q-Learning

- Học thử-sai gần với cách con người học, gần với lý tưởng trong nhiều bài toán.
- **Off-policy:** không bị ràng buộc chặt một policy cố định, có thể tối ưu tới policy tốt nhất.
- **Model-free:** linh hoạt khi tham số môi trường không mô tả được động.
- Có thể **sửa lỗi** trong quá trình huấn luyện; xác suất lặp lại cùng sai lầm thường thấp.

Nhược điểm Q-Learning

- Khó tìm cân bằng giữa **thử hành động mới** và **giữ hành động đã biết tốt**.

- Đôi khi **lạc quan quá mức**, overestimate chất lượng hành động hoặc chiến lược.
- Với nhiều lựa chọn giải pháp, có thể **tốn thời gian** mới xác định được chiến lược tối ưu.

Ứng dụng Q-Learning

- **Trò chơi**: học chiến lược đạt trình độ cao trong nhiều game.
- **Hệ gợi ý**: cải thiện hệ thống gợi ý (ví dụ nền tảng quảng cáo).
- **Robot**: thao tác vật thể, tránh vật cản, vận chuyển.
- **Xe tự lái**: quyết định như đổi làn, dừng xe.
- **Chuỗi cung ứng**: tối ưu đường đưa sản phẩm ra thị trường.

4 Thuật toán REINFORCE

REINFORCE Algorithm

REINFORCE là thuật toán **policy gradient** trong RL, dựa trên phương pháp **Monte Carlo**. Cách triển khai đơn giản: dùng **gradient ascent** để tăng trực tiếp phần thưởng tích lũy kỳ vọng bằng cách điều chỉnh tham số policy. Thuật toán **không** cần mô hình môi trường $\Rightarrow$ thuộc nhóm **model-free**.

4.1 Khái niệm chính

Policy Gradient Methods

REINFORCE thuộc lớp **policy gradient**: cải thiện policy bằng cách đi theo **gradient** của phần thưởng tích lũy kỳ vọng đối với tham số policy.

Monte Carlo trong REINFORCE

REINFORCE là dạng Monte Carlo vì dùng **lấy mẫu (sampling)** để ước lượng các đại lượng mong muốn (return, gradient) từ **toàn bộ episode**, không bootstrap từng bước như TD.

4.2 Cách REINFORCE hoạt động

Lịch sử và mục tiêu

Thuật toán do Ronald J. Williams giới thiệu (1992). Mục tiêu: **tối đa hóa phần thưởng tích lũy kỳ vọng** bằng cách điều chỉnh tham số policy; agent học ra quyết định **tự** trong môi trường.

4.2.1 Lấy mẫu episode (Episode Sampling)

Bước 1 — Lấy mẫu episode

Bắt đầu bằng cách lấy mẫu một **episode đầy đủ**: agent tương tác môi trường theo **policy hiện tại**. Episode là chuỗi trạng thái, hành động và reward cho đến khi đạt **trạng thái kết thúc**.

4.2.2 Quỹ đạo (Trajectory)

Bước 2 — Ghi quỹ đạo

Agent ghi lại quỹ đạo tương tác:

$$(s_1, a_1, r_1), (s_2, a_2, r_2), \dots, (s_t, a_t, r_t)$$

trong đó s là trạng thái, a là hành động, r là reward tại mỗi bước.

4.2.3 Return G_t

Bước 3 — Tính return

Return G_t là **phần thưởng tích lũy kỳ vọng** từ thời điểm t trở đi:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

với γ là hệ số chiết khấu.

4.2.4 Policy gradient

Bước 4 — Gradient của return kỳ vọng

Tính gradient của return kỳ vọng theo tham số θ của policy π_θ . Theo nguyên lý policy gradient (Monte Carlo), cập nhật dựa trên **log-likelihood** của chuỗi hành động đã chọn:

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$$

4.2.5 Cập nhật policy

Bước 5 — Cập nhật tham số policy

Sau khi có gradient, cập nhật tham số theo hướng **tăng** reward kỳ vọng, ví dụ:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$$

với α là learning rate.

So với TD (Q-learning, SARSA)

Học TD (Q-learning, SARSA) thường nhấn mạnh reward **tức thời** và bootstrap. REINFORCE cho agent học từ **toàn bộ chuỗi** trạng thái, hành động và reward của episode. Lặp các bước trên cho đến khi episode (hoặc quá trình huấn luyện) kết thúc theo tiêu chí dừng.

4.3 Ưu và nhược điểm

Ưu điểm REINFORCE

- **Model-free**: phù hợp khi môi trường không biết hoặc khó mô hình hóa.
- **Đơn giản, trực quan**: dễ hiểu và triển khai.
- Xử lý được **không gian hành động chiều cao hoặc liên tục** — khác nhiều phương pháp dựa trên giá trị rời rạc.

Nhược điểm REINFORCE

- **Phương sai cao**: ước lượng gradient biến động mạnh, học có thể chậm và **không ổn định**.
- **Dùng mẫu kém hiệu quả**: mỗi lần tính gradient thường cần bộ mẫu episode mới, kém tái sử dụng mẫu hơn các kỹ thuật dùng lại dữ liệu nhiều lần.

5 SARSA (State-Action-Reward-State-Action)

SARSA

SARSA (State-Action-Reward-State-Action) là biến thể của Q-learning trong đó **chính sách mục tiêu trùng chính sách hành vi** (on-policy). Hai cặp trạng thái-hành động liên tiếp (s, a) và (s', a') cùng phần thưởng tức thời r khi chuyển từ s sang s' quyết định giá trị cập nhật Q — đó là lý do tên thuật toán.

Ý tưởng học thử-sai

SARSA là kỹ thuật học **on-policy** hiệu quả: agent thực hiện hành động, quan sát hậu quả, rồi điều chỉnh kế hoạch theo kết quả. Cốt lõi là **thử và sai**: chọn hành động trong tình huống, nhận phản hồi từ môi trường, cập nhật chiến lược dần theo chuỗi S-A-R-S-A.

5.1 Ví dụ robot trong mê cung

Robot đi mê cung

Giả sử dạy robot đi qua mê cung. Vị trí hiện tại là **trạng thái** s ; mục tiêu tìm đường tốt tới đích. Mỗi bước robot có thể **hành động** (đi lên, xuống, trái, phải). Môi trường trả **phần thưởng** dương hoặc âm để báo hiệu bước đi tốt hay tệ. Sau khi thực hiện a tại s , robot chuyển sang **trạng thái kế** s' ; hành động **tiếp theo thực sự** a' (theo chính sách hiện tại, ví dụ ϵ -greedy) sẽ xuất hiện trong công thức cập nhật SARSA — khác với Q-learning dùng $\max_{a'} Q(s', a')$.

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

The diagram shows the SARSA update formula with four terms labeled below them using green brackets:

- $Q(s,a)$ (left): Updated Q-value
- $Q(s,a)$ (middle): Current Q-value
- $r + \gamma Q(s',a')$ (middle): Target Q-value
- $Q(s,a)$ (right): Current Q-value

5.2 Phương trình cập nhật

Công thức SARSA

Giá trị $Q(s, a)$ được cập nhật theo mẫu TD on-policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

trong đó α là tốc độ học, γ hệ số chiết khấu, r phần thưởng nhận được sau khi thực hiện a tại s , s' trạng thái kế, và a' là **hành động thực sự** sẽ chọn tại s' theo chính sách hành vi (không phải hành động tối ưu giả định).

Đọc nhanh công thức

Mục tiêu TD là $r + \gamma Q(s', a')$: phần thưởng tức thời cộng giá trị chiết khấu của cặp trạng thái–hành động **kế tiếp theo chính sách đang theo**. Sai số $r + \gamma Q(s', a') - Q(s, a)$ nhân α rồi cộng vào $Q(s, a)$.

5.3 Các thành phần cốt lõi

Trạng thái S

Trạng thái phản ánh môi trường, chứa mọi thông tin về tình huống hiện tại của agent (ví dụ ô trong mê cung).

Hành động A

Hành động là quyết định của agent trong trạng thái hiện tại; thực hiện a gây chuyển từ s sang trạng thái khác.

Phần thưởng R

Phần thưởng là tín hiệu phản hồi tức thời từ môi trường sau hành động tại s ; hướng agent biết hành động nào được ưu tiên trong bối cảnh đó.

Trạng thái kế S'

Sau khi thực hiện a tại s , agent vào trạng thái mới s' — môi trường đã thay đổi theo hành động vừa chọn.

5.4 Cách SARSA hoạt động**Các bước thuật toán SARSA**

1. **Khởi tạo bảng Q** : gán $Q(s, a)$ giá trị tùy ý (hoặc 0); chọn trạng thái ban đầu s và hành động ban đầu a (thường bằng ϵ -greedy trên Q hiện tại).
2. **Khai thác vs khám phá**: khai thác dùng Q đã biết để tối đa phần thưởng ngắn hạn; khám phá thử hành động ít chắc chắn để tìm chiến lược tốt hơn về lâu dài.
3. **Thực thi và phản hồi**: thực hiện a , nhận r , chuyển sang s' .
4. **Cập nhật Q** : chọn a' tại s' theo chính sách hành vi; cập nhật $Q(s, a)$ bằng công thức SARSA ở trên.
5. **Lặp**: đặt $s \leftarrow s'$, $a \leftarrow a'$; lặp đến khi episode kết thúc. Qua nhiều vòng, Q tiệm cận giá trị kỳ vọng theo **chính sách đang theo**, giúp quyết định ổn định hơn trong môi trường rủi ro.

Đặc điểm on-policy

Mỗi bước học phụ thuộc hành động **thực sự** sẽ làm ở s' , nên khám phá ảnh hưởng trực tiếp tới cập nhật — phù hợp khi cần học an toàn, ổn định theo chính sách triển khai (y tế, giao thông, học cá nhân hóa).

5.5 SARSA so với Q-learning

Bảng so sánh

Tiêu chí	SARSA	Q-learning
Loại chính sách	On-policy (mục tiêu = hành vi).	Off-policy (học tối ưu trong khi hành vi có thể khác).
Quy tắc cập nhật	$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$	$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$
Hội tụ	Thường chậm hơn tối chính sách tối ưu toàn cục nhưng bám sát chính sách an toàn.	Thường nhanh hơn tối Q^* nếu đủ khám phá.
Khám phá-khai thác	Khám phá ảnh hưởng trực tiếp bước học.	Có thể tách chính sách khám phá khỏi chính sách học.
Cập nhật policy	Dựa trên hành động đã thực sự chọn ở s' .	Giả định hành động tốt nhất ở s' ($\max_{a'}$).
Ứng dụng	Môi trường cần ổn định, rủi ro cao khi lệch chính sách.	Game, robot, giao dịch khi ưu tiên hiệu quả tối đa.

Chọn thuật toán

Dùng SARSA khi muốn học giá trị của **chính sách đang chạy** (kể cả ϵ -greedy). Dùng Q-learning khi muốn ước lượng Q^* của chính sách tối ưu trong khi vẫn khám phá bằng chính sách khác.

6 Phương pháp Actor–Critic

Actor–Critic

Thuật toán **actor–critic** kết hợp học **dựa trên chính sách** (policy-based) và học **dựa trên giá trị** (value-based) để khắc phục hạn chế khi chỉ dùng một nhóm kỹ thuật. **Actor** chọn hành động theo chính sách; **Critic** đánh giá hành động đó bằng hàm giá trị, tạo phản hồi cho actor cải thiện chính sách trong khi cân bằng khám phá-khai thác.

6.1 Hai thành phần chính

Actor

Actor chịu trách nhiệm chọn hành động theo chính sách hiện tại, thường ký hiệu $\pi_\theta(a | s)$ — xác suất (hoặc phân phối) chọn a khi ở s , với tham số θ .

Critic

Critic ước lượng hàm giá trị $V(s)$ hoặc $Q(s, a)$ — kỳ vọng phần thưởng tích lũy từ s (hoặc từ cặp (s, a)) — để đo chất lượng hành động của actor.

6.2 Cách hoạt động từng bước

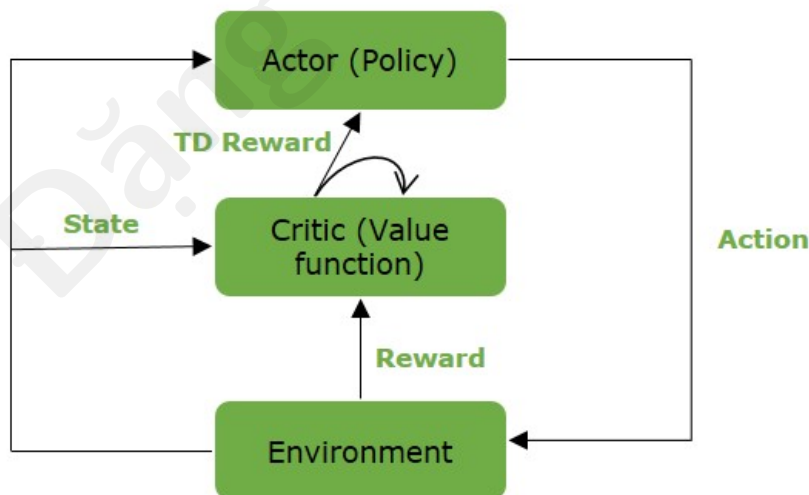
Mục tiêu

Học chính sách tối đa hóa phần thưởng kỳ vọng tích lũy: actor đề xuất hành động, critic chấm điểm, phản hồi dùng để cập nhật cả θ (policy) và trọng số critic.

Giải thuật (pseudo)

1. Khởi tạo tham số chính sách actor θ , hàm giá trị critic, môi trường; chọn trạng thái ban đầu s_0 .
2. Lấy mẫu (s_t, a_t) theo π_θ từ mạng actor.
3. Tính **hàm lợi thế** (advantage), thường qua **sai số TD** $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ do critic cung cấp.
4. Tính gradient chính sách (policy gradient) có trọng số bởi δ_t hoặc $A(s_t, a_t)$.
5. Cập nhật θ theo hướng tăng phần thưởng kỳ vọng.
6. Cập nhật trọng số critic (học giá trị) từ δ_t .
7. Lặp các bước trên cho tới khi chính sách hội tụ hoặc đạt tiêu chí dừng.

Actor-Critic Working



6.3 Ưu điểm

Ưu điểm actor–critic

- **Hiệu quả mẫu:** kết hợp policy và value giúp cần ít tương tác môi trường hơn so với chỉ policy gradient thuần.
- **Hội tụ nhanh hơn:** cập nhật đồng thời chính sách và giá trị, thích nghi nhanh tác vụ mới.
- **Linh hoạt không gian hành động:** xử lý hành động rời rạc và liên tục (với biến thể như DDPG, SAC).
- **Off-policy (một số biến thể):** có thể học từ kinh nghiệm cũ không khớp chính sách hiện tại (SAC, DDPG).

6.4 Thách thức

Thách thức cần xử lý

- **Phương sai cao:** ước lượng gradient policy vẫn nhiều; giảm bằng GAE (Generalized Advantage Estimation) hoặc baseline tốt hơn.
- **Ổn định huấn luyện:** actor và critic học đồng thời dễ lệch pha; dùng TRPO, PPO để giới hạn bước cập nhật policy.
- **Đánh đổi bias–variance:** critic kém chính xác gây bias gradient; critic quá nhạy gây dao động — cần tinh chỉnh tốc độ học và kiến trúc mạng.

6.5 Các biến thể tiêu biểu

A2C — Advantage Actor-Critic

A2C dùng **hàm lợi thế** $A(s, a) = Q(s, a) - V(s)$ (hoặc xấp xỉ qua n -step return) để hướng gradient tới hành động tốt hơn trung bình tại s . Huấn luyện **đồng bộ**, thường một hoặc vài worker, cập nhật model sau mỗi batch bước — dễ triển khai.

A3C — Asynchronous Advantage Actor-Critic

A3C (Mnih et al., 2016) chạy **nhiều worker song song**, mỗi worker có mạng cục bộ, thu thập kinh nghiệm và đẩy gradient **bất đồng bộ** lên mạng global — giảm tương quan mẫu, ổn định và nhanh hơn trên môi trường phức tạp.

SAC — Soft Actor-Critic

SAC là off-policy, thêm **điều hòa entropy** vào mục tiêu để khuyến khích khám phá; tối ưu đồng thời return kỳ vọng và độ “mềm” của chính sách — cân bằng khai thác–khám phá tốt trên hành động liên tục.

DDPG — Deep Deterministic Policy Gradient

DDPG cho **không gian hành động liên tục**: actor xuất hành động xác định, critic xấp xỉ Q ; dùng **mạng target** và replay buffer để ổn định huấn luyện.

Q-Prop

Q-Prop giảm phương sai gradient bằng **control variate** từ critic, không thêm bias như một số mẹo TD thuần — kết hợp policy gradient với ước lượng Q chính xác hơn.

6.6 Thuật toán A2C**Các bước A2C**

1. Khởi tạo tham số policy θ , tham số value w , và môi trường.
2. Agent tương tác môi trường: chọn $a_t \sim \pi_\theta(\cdot | s_t)$, nhận r_t , quan sát s_{t+1} .
3. Tính $A(s_t, a_t)$ từ rollout (hoặc n -step) và $V_w(s_t)$.
4. Cập nhật actor: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t | s_t) A(s_t, a_t)$.
5. Cập nhật critic: giảm sai số TD hoặc MSE giữa V_w và return mục tiêu.
6. Lặp đến hội tụ; có thể dùng nhiều môi trường song song nhưng **đồng bộ** hóa gradient trước mỗi bước global.

6.7 Thuật toán A3C**Các bước A3C**

1. Khởi tạo **mạng global** (policy + value).
2. Khởi chạy nhiều **worker** song song; mỗi worker sao chép mạng global thành mạng cục bộ.
3. Mỗi worker: quan sát s , chọn a theo policy cục bộ, nhận r và s' ; tính advantage / δ .
4. Worker cập nhật critic và actor cục bộ trên đoạn trajectory ngắn ($t_{\max}$ bước).
5. Worker gửi gradient lên global network; global cập nhật **bất đồng bộ** (không chờ worker khác).
6. Worker đồng bộ lại tham số từ global và tiếp tục thu thập kinh nghiệm.

6.8 A2C so với A3C

Bảng so sánh A2C và A3C

Tiêu chí	A2C	A3C
Song song	Thường ít worker; cập nhật đồng bộ sau khi gom gradient.	Nhiều worker; cập nhật global bất đồng bộ .
Cập nhật mô hình	Gradient từ batch worker được tổng hợp rồi áp dụng một lần.	Mỗi worker đẩy gradient độc lập, không đợi nhau.
Tốc độ học	Gradient descent chuẩn; có thể hội tụ nhanh nhưng dễ dao động nếu batch nhỏ.	Cập nhật rải rác, đôi khi ổn định hơn nhờ giảm tương quan.
Ổn định	Phụ thuộc đồng bộ hóa; dễ lệch nếu critic yếu.	Thường ổn định hơn nhờ nhiều luồng khám phá khác nhau.
Hiệu quả lấy mẫu	Một (hoặc ít) luồng môi trường.	Nhiều luồng song song — thu mẫu nhanh hơn.
Triển khai	Đơn giản, phù hợp GPU/CPU vừa.	Phức tạp hơn (quản lý tiến trình, đồng bộ tham số).
Tốc độ hội tụ	Chậm hơn nếu chỉ một agent học liên tục.	Nhanh hơn khi nhiều vùng môi trường được khám phá song song.
Chi phí tính toán	Thấp hơn.	Cao hơn (nhiều worker).
Use case	Môi trường đơn giản, tài nguyên hạn chế.	Môi trường lớn, cần khám phá mạnh và song song hóa.

7 Phương pháp Monte Carlo trong RL

Monte Carlo trong học tăng cường

Phương pháp Monte Carlo (MC) học trực tiếp từ các **episode** trải nghiệm khi agent tương tác môi trường, **không cần** biết trước mô hình chuyển trạng thái MDP. Ý tưởng: dùng lấy mẫu ngẫu nhiên lặp lại để ước lượng số học các giá trị khó tính giải tích (giá trị trạng thái, hành động, hoặc cặp trạng thái–hành động).

7.1 Khái niệm then chốt

Episode

Chuỗi trạng thái, hành động và phần thưởng từ lúc bắt đầu đến khi **kết thúc** episode (đích, thua, hoặc hết giới hạn bước).

Return G_t

Tổng phần thưởng tích lũy (thường có chiết khấu γ) từ bước t trở về sau trong một episode:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Hàm giá trị V

Hàm dự đoán phần thưởng kỳ vọng khi bắt đầu từ trạng thái s (hoặc từ cặp (s, a) với Q).

7.2 Đánh giá chính sách Monte Carlo**Ước lượng trung bình return**

MC ước lượng $V(s)$ bằng cách **lấy trung bình** return quan sát được mỗi khi s xuất hiện trong nhiều episode. Với N episode có ghé s , return thứ i là G_i :

$$V(s) = \frac{1}{N} \sum_{i=1}^N G_i$$

trong đó i là chỉ số episode, s là trạng thái cần ước lượng, N số lần s được thăm, G_i tổng phần thưởng chiết khấu từ lần thăm s trong episode i trở đi.

Quy trình ước lượng từng bước

1. **Tạo episode:** agent theo chính sách π tương tác môi trường, sinh chuỗi (s, a, r) .
2. **Tính return:** với mỗi s (hoặc (s, a)) xuất hiện, tính G_t từ thời điểm đó đến hết episode.
3. **Cập nhật giá trị:** ghi nhận các G ; cập nhật $V(s)$ (hoặc $Q(s, a)$) bằng trung bình mẫu (first-visit hoặc every-visit).
4. Lặp nhiều episode cho đến khi ước lượng ổn định.

7.3 On-policy và off-policy**On-policy**

Chính sách sinh episode trùng chính sách đang đánh giá hoặc cải thiện. Agent học từ hành vi **do chính mình** gây ra. Ví dụ: **first-visit MC** — chỉ dùng return lần **đầu tiên** s xuất hiện trong mỗi episode để cập nhật $V(s)$.

Off-policy

Episode có thể sinh bởi **chính sách hành vi** khác chính sách **mục tiêu** đang tối ưu; học từ quỹ đạo do policy khác tạo. Cần **trọng số importance sampling** để chỉnh cập nhật cho đúng phân phối mục tiêu.

7.4 Điều khiển Monte Carlo (MC control)**Khám phá trong MC**

Cần cân bằng khám phá-khai thác: dùng ε -greedy hoặc softmax để đảm bảo mọi (s, a) được thăm đủ, tránh chính sách kẹt cục bộ.

Cải thiện chính sách qua Q

MC control lặp: ước lượng $Q(s, a)$ — phần thưởng kỳ vọng khi ở s , chọn a , rồi theo π — và **cải thiện** π bằng cách chọn hành động cho $Q(s, a)$ **lớn nhất** (greedy trên Q), có thể kèm ε -greedy để khám phá.

Thuật toán MC control (generating traces)

1. Khởi tạo $Q(s, a)$ tùy ý và chính sách $\pi(s)$ (ví dụ ε -soft).
2. Với mỗi episode: sinh chuỗi theo π (có khám phá).
3. Tính G_t cho mỗi cặp (s, a) xuất hiện (first-visit hoặc every-visit).
4. Cập nhật:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

5. **Cải thiện chính sách:** $\pi(s) \leftarrow \arg \max_a Q(s, a)$ (hoặc ε -greedy trên Q).
6. Lặp đến khi π và Q hội tụ gần tối ưu.

Sửa nhầm “minimize Q ”

Khi cải thiện chính sách phải chọn hành động **maximize** $Q(s, a)$, không phải minimize — maximize mới hướng tới phần thưởng cao nhất tại mỗi s .

7.5 Ứng dụng**Ứng dụng điển hình**

- **Trò chơi:** cờ, bài, game có episode rõ ràng và reward cuối ván.
- **Robot:** học điều hướng, thao tác từ trial thực tế khi không có mô hình động học chính xác.
- **Tài chính:** mô phỏng giá, định giá quyền chọn, tối ưu danh mục khi mô hình phân

tích khó.

7.6 Hạn chế

Hạn chế Monte Carlo

- **Phương sai cao:** ít episode thì ước lượng dao động mạnh giữa các lần chạy.
- **Episode dài:** phải chờ hết episode mới cập nhật — kém hiệu quả khi reward trễ hoặc chuỗi rất dài.
- **Không bootstrap:** không dùng ước lượng khác để chỉnh ước lượng hiện tại (khác TD) — học chậm hơn trên không gian trạng thái lớn.

8 Học chênh lệch thời gian (Temporal Difference)

Temporal Difference (TD) learning

Học chênh lệch thời gian (TD) là kỹ thuật RL **không mô hình** (model-free): căn chỉnh dự đoán hiện tại theo dự đoán **kế tiếp** đã bootstrap, kết hợp phần thưởng tức thời và giá trị trạng thái sau. Tỉn hiệu huấn luyện đến từ **dự đoán tương lai**, không cần chờ hết episode như Monte Carlo.

Vị trí giữa MC và DP

Monte Carlo chỉ cập nhật sau khi biết **kết quả cuối** episode; Dynamic Programming cần mô hình chuyển trạng thái. TD học từ kinh nghiệm thực như MC nhưng **bootstrap** từ ước lượng V hoặc Q như DP — cập nhật từng bước, phù hợp bài toán liên tục và không kết thúc.

8.1 Tham số α , γ , ε

α — tốc độ học

$\alpha \in [0, 1]$: mức điều chỉnh ước lượng theo sai số TD. α lớn — thích nghi nhanh nhưng nhiều; α nhỏ — ổn định nhưng chậm.

γ — hệ số chiết khấu

$\gamma \in [0, 1]$: trọng số phần thưởng tương lai. γ gần 1 — coi trọng dài hạn; γ nhỏ — ưu tiên phần thưởng gần.

ε — khám phá

Với ε -greedy: xác suất ε chọn hành động ngẫu nhiên, $1 - \varepsilon$ chọn hành động tốt nhất theo ước lượng hiện tại. ε lớn — khám phá nhiều hơn trong huấn luyện.

8.2 TD trong AI và học máy**Vai trò thực tiễn**

TD là nền cho robot, game, tài chính thời gian thực: cập nhật giá trị từng bước khi reward trễ, thích nghi khi chuỗi không có điểm kết thúc rõ. Kết hợp quan sát và kỳ vọng tương lai giúp xây agent thích nghi nhanh.

8.3 Thuật toán TD(λ)**TD(λ)**

TD(λ) trộn các return n -bước với trọng số λ :

- $\lambda = 0$: tương đương **TD(0)** — một bước bootstrap.
- $\lambda = 1$: gần **Monte Carlo** — cập nhật theo return đầy đủ tới hết episode.
- $0 < \lambda < 1$: trộn phần thưởng ngắn hạn (TD) và dài hạn (MC), thường qua **eligibility traces**.

8.4 TD(0)**Quy tắc TD(0) (one-step)**

Cập nhật hàm giá trị trạng thái:

$$V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

- $V(s_t)$: ước lượng hiện tại tại s_t .
- R_{t+1} : phần thưởng sau khi rời s_t .
- $V(s_{t+1})$: bootstrap giá trị trạng thái kế.
- α, γ : tốc độ học và chiết khấu.

Sai số trong ngoặc là mục tiêu TD một bước trừ ước lượng cũ.

8.5 TD(1)

TD(1)

TD(1) (trace length 1) là dạng **tổng quát** của TD(0), trộn MC và DP: điều chỉnh V dùng phần thưởng vừa nhận và chuỗi dự đoán phần thưởng tương lai qua eligibility trace — nhấn mạnh credit assignment dọc theo chuỗi hành động gần đây.

8.6 TD so với Q-learning

Bảng so sánh

Khía cạnh	TD (giá trị trạng thái)	Q-learning
Mục tiêu	Ước lượng $V(s)$.	Ước lượng $Q(s, a)$.
Đối tượng	Giá trị trạng thái.	Giá trị cặp trạng thái–hành động.
Chính sách	Model-free; on- hoặc off-policy tùy biến thể.	Model-free; off-policy (học max Q).
Quy tắc cập nhật	Dựa trên $V(s_{t+1})$ của bước kế.	Dựa trên $\max_{a'} Q(s_{t+1}, a')$.
Công thức	$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$	$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$
Khám phá	Theo trade-off của chính sách hiện tại (ϵ -greedy).	Có thể tách policy khám phá khỏi policy học tối ưu.
Hội tụ	Xấp xỉ V^π của chính sách đang đánh giá.	Tới Q^* nếu đủ khám phá.
Ví dụ	TD(0), SARSA (cập nhật Q on-policy).	Q-learning.

8.7 Sai số TD (TD error)

Định nghĩa TD error

Sai số TD tại bước t là chênh lệch giữa **mục tiêu bootstrap** và ước lượng hiện tại:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Cần thông tin bước $t + 1$ nên δ_t chỉ dùng được sau khi quan sát s_{t+1} và r_{t+1} . Cập nhật V bằng δ_t gọi là **backup** một bước; δ_t liên hệ phương trình Bellman cho V .

TD error với Q

Với hàm Q , dạng tương tự: $\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$ (SARSA) hoặc thay $Q(s_{t+1}, a_{t+1})$ bằng $\max_{a'} Q(s_{t+1}, a')$ trong Q-learning.

8.8 Lợi ích

Lợi ích TD

- Học từ chuỗi **chưa hoàn thành** — áp dụng bài toán liên tục.
- Hoạt động khi môi trường **không có trạng thái kết thúc**.
- Phương sai thường **thấp hơn MC** vì cập nhật từng bước, bootstrap một transition thay vì cả episode.

8.9 Thách thức

Thách thức TD

- **Nhảy giá trị khởi tạo**: V hoặc Q ban đầu ảnh hưởng mạnh quỹ đạo học sớm.
- **Bias do bootstrap**: ước lượng dựa trên ước lượng khác gây thiên lệch, đổi lấy phương sai thấp hơn MC thuần.
- Cần tinh chỉnh α , γ và lịch giảm ϵ để hội tụ ổn định.